🖥️ Fubo Senior SRE — One-Page Cheat Sheet

Unix · K8s · Terraform · AWS · CI/CD — pure quick-reference, no explanations

UNIX — PROCESS & SYSTEM

`ps aux` / `ps -ef`
`top` / `htop`
`kill -15 PID` graceful · `kill -9 PID` force
`killall <name>`
`nohup ./x.sh &`
`systemctl status/start/stop/restart <svc>`
`journalctl -u <svc> -f`
`df -hP` disk usage
`du -h --max-depth=1 | sort -rh`
`free -h` · `uptime` · `vmstat 1` · `iostat -x 1`
`lsof -i :PORT` · `lsof -p PID`

UNIX — NETWORKING

`ip a`
`ss -tulnp`
`curl -I <url>`
`curl -o /dev/null -s -w "%{http_code}" url`
`dig +short <domain>`
`traceroute host` / `mtr host`
`nc -zv host port` port check
`tcpdump -i eth0 port 443`
`iptables -L -n`
Key files: /etc/hosts, /etc/resolv.conf

UNIX — TEXT & PERMISSIONS

`grep -rniv "pat" path`
`awk -F: '{print $1}' f`
`sed -i 's/old/new/g' f`
`sort f | uniq -c | sort -rn` top freq
`tail -f f` · `less +F f`
`cut -d',' -f2 f.csv`
`chmod 755` / `chown u:g f` / `umask`

DOCKER

`docker ps -a`
`docker logs -f <c>`
`docker exec -it <c> sh`
`docker build -t name:tag .`
`docker run -d -p 8080:80 --name x img`
`docker stats`
`docker system prune -a`
ENTRYPOINT = fixed exec · CMD = default args (overridable)

KUBERNETES — DEBUG

`kubectl get pods -o wide -n ns`
`kubectl describe pod <p> -n ns`
`kubectl logs <p> [-c c] [--previous] [-f]`
`kubectl exec -it <p> -- sh`
`kubectl get events --sort-by=.metadata.creationTimestamp`
`kubectl top pod` / `top node`
`kubectl rollout status|history|undo deploy/n`
`kubectl scale deploy/n --replicas=N`
`kubectl port-forward pod/p 8080:80`
`kubectl get pods -A -o wide | grep -v Running`
`kubectl apply -f f.yaml --dry-run=client -o yaml`
`kubectl explain <resource>`
`kubectl auth can-i <verb> <resource>`

TERRAFORM — CLI

`terraform init / validate / fmt`
`terraform plan -out=tfplan`
`terraform apply tfplan`
`terraform destroy`
`terraform state list`
`terraform state show <r>`
`terraform state mv/rm <r>`
`terraform import <r> <id>`
`terraform apply -replace=<r>`
`terraform workspace list/new/select`

GIT

`git status / diff / log --oneline --graph`
`git switch -c <b>`
`git rebase -i HEAD~3`
`git reset --soft|--hard HEAD~1`
`git revert <c>` safe on shared history
`git stash` / `git stash pop`
`git bisect start`
merge = preserves history · rebase = linear, local only

AWS — NETWORKING

VPC	isolated net, CIDR block
Public subnet	route → IGW
Private subnet	route → NAT GW
SG	stateful, allow-only, instance-level
NACL	stateless, allow+deny, subnet, ordered
Peering	non-transitive VPC↔VPC
Transit GW	transitive hub-and-spoke
VPC Endpoint	Gateway(S3/DynamoDB,free) / Interface(PrivateLink)
ALB / NLB	L7 HTTP routing / L4 raw+static IP

AWS — ARCHITECTURE PATTERNS

```
HA 3-tier (2 AZ):
Route53 -> ALB (2 AZ)
  AZ-a app  |  AZ-b app
     \       /
  ElastiCache (Redis)
        |
  RDS Aurora Multi-AZ
  Primary(a) <-> Standby(b)

EKS subnets:
VPC 10.0.0.0/16
 Public  (IGW,ALB,NAT/AZ)
 Private (EKS nodes,RDS,
        out via NAT only)
```

CI/CD — STEPS IN ORDER

1. PR → lint/SAST
2. unit tests
3. build image
4. Trivy scan (fail on critical)
5. push registry, tag=SHA
6. update GitOps repo (Helm/Kustomize)
7. ArgoCD detects & syncs
8. canary % traffic shift
9. Prometheus-backed analysis
10. promote 100% or auto-rollback
11. smoke tests
12. deploy annotation on dashboard

DOCKERFILE — MULTI-STAGE BUILD

```
# ---- Build stage ----
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

# ---- Final stage: no build tools, no root ----
FROM python:3.12-slim
WORKDIR /app

RUN useradd --create-home --uid 1000 appuser
COPY --from=builder /root/.local /home/appuser/.local
COPY --chown=appuser:appuser . .

ENV PATH=/home/appuser/.local/bin:$PATH \
    PYTHONUNBUFFERED=1

USER appuser
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s \
  CMD curl -f http://localhost:8080/health || exit 1

ENTRYPOINT ["python", "-m", "app"]
```

Why multi-stage: build tools (compilers, pip cache) never ship in the final image — smaller image, smaller attack surface. Non-root `USER appuser` + `--chown` on copy is the part interviewers listen for.

For a **Go** service the same principle goes further: `FROM golang:1.22 AS builder` → compile a static binary → `COPY --from=builder /app/bin /bin/app` into `FROM scratch` or `gcr.io/distroless/static`. No shell, no package manager, often <20MB final image.

CI/CD PIPELINE — GITHUB ACTIONS + ARGOCD (GITOPS)

```
git push main
  |
GitHub Actions:  test -> build -> Trivy scan -> push image (tag=SHA)
  |
bump image tag in GitOps repo  (git commit + push)
  |
ArgoCD detects the diff (polls or webhook)
  |
auto-sync: prune + selfHeal  -->  K8s cluster
  |
canary rollout  -->  deploy annotation added to dashboard
```

.GITHUB/WORKFLOWS/DEPLOY.YML

```yaml
name: build-and-deploy
on:
  push:
    branches: [main]

permissions:
  id-token: write   # OIDC, no static AWS keys
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run tests
        run: pytest --maxfail=1

      - name: Configure AWS creds (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gh-actions-deploy
          aws-region: us-east-1

      - name: Build, scan, push image
        run: |
          aws ecr get-login-password | \
            docker login --username AWS --password-stdin $ECR_REPO
          docker build -t $ECR_REPO:$GITHUB_SHA .
          trivy image --exit-code 1 \
```

ARGOCD-APP.YAML

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: adserver
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/fubo/gitops-repo.git
    targetRevision: main
    path: apps/adserver
    helm:
      valueFiles:
        - values.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: adtech
  syncPolicy:
    automated:
      prune: true       # remove what's no longer in git
      selfHeal: true    # revert manual kubectl drift
    syncOptions:
      - CreateNamespace=true
    retry:
      limit: 3
      backoff:
        duration: 10s
        maxDuration: 3m
```

```
            --severity CRITICAL,HIGH $ECR_REPO:$GITHUB_SHA
          docker push $ECR_REPO:$GITHUB_SHA

      - name: Bump tag in GitOps repo
        run: |
          git clone https://x-access-token:${{ secrets.GITOPS_TOKEN }}@github.com/fubo/gitops-repo.git
          cd gitops-repo
          yq -i '.image.tag = strenv(GITHUB_SHA)' apps/adserver/values.yaml
          git commit -am "deploy adserver@$GITHUB_SHA" && git push
```

CI (GitHub Actions) stops at "update the GitOps repo." CD (ArgoCD) takes over from there — that boundary is the whole point of GitOps: nothing ever pushes into the cluster directly.

DEPLOYMENT.YAML — PRODUCTION-GRADE

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: adserver
  namespace: adtech
  labels: { app: adserver }
spec:
  replicas: 6
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0     # never drop below desired capacity
      maxSurge: 25%
  selector:
    matchLabels: { app: adserver }
  template:
    metadata:
      labels: { app: adserver }
    spec:
      serviceAccountName: adserver-sa       # IRSA-bound for AWS access
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels: { app: adserver }
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        fsGroup: 1000
      containers:
        - name: adserver
          image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/adserver:GITHUB_SHA
          ports:
            - containerPort: 8080
          resources:
            requests: { cpu: "500m", memory: "512Mi" }
            limits:   { cpu: "500m", memory: "512Mi" }   # requests=limits -> Guaranteed QoS
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
            initialDelaySeconds: 5
```

```
            periodSeconds: 10
          livenessProbe:
            httpGet: { path: /health, port: 8080 }
            initialDelaySeconds: 15
            periodSeconds: 20
            failureThreshold: 3
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: adserver-pdb
  namespace: adtech
spec:
  minAvailable: 80%
  selector:
    matchLabels: { app: adserver }
```

Five things an interviewer scans this for: **maxUnavailable: 0** (zero-downtime), **requests=limits** (Guaranteed QoS), **topologySpreadConstraints** (real multi-AZ spread, not just replica count), **non-root + readOnlyRootFilesystem** (security posture), and the standalone **PodDisruptionBudget** — a Deployment alone doesn't protect you during a node drain, the PDB is what does.

SLI / SLO / ERROR BUDGET · GOLDEN SIGNALS

SLI = the measured indicator (e.g. success rate)
SLO = the target (e.g. 99.9%)
Error budget = 100% − SLO (0.1%) = allowed "badness"
Burn rate = speed of budget consumption vs sustainable pace

Golden signals: Latency (p95/p99) · Traffic · Errors · Saturation

HTTP & EXIT CODES

502 upstream down/bad response
503 overloaded / health check failing
504 upstream too slow / timeout
429 rate limited

Exit codes: 0 ok · 1 general error · 126 not executable · 127 not found · 137 SIGKILL/OOM · 143 SIGTERM

ONE TERRAFORM MODULE — VPC WITH PUBLIC/PRIVATE SUBNETS (HA NAT PER AZ)

VARIABLES.TF

```
variable "name"     { type = string }
variable "vpc_cidr" { type = string
  default = "10.0.0.0/16" }
variable "azs" {
  type = list(string)          # ["us-east-1a","us-east-1b"]
}
variable "public_subnet_cidrs"  { type = list(string) }
variable "private_subnet_cidrs" { type = list(string) }
```

OUTPUTS.TF

```
output "vpc_id"             { value = aws_vpc.this.id }
output "public_subnet_ids"  {
  value = [for s in aws_subnet.public  : s.id] }
output "private_subnet_ids" {
  value = [for s in aws_subnet.private : s.id] }
```

USAGE

```
module "vpc" {
  source  = "./modules/vpc"
  name    = "fubo-adtech"
  azs     = ["us-east-1a", "us-east-1b"]
  public_subnet_cidrs  = ["10.0.1.0/24","10.0.2.0/24"]
  private_subnet_cidrs = ["10.0.11.0/24","10.0.12.0/24"]
}
```

MAIN.TF

```
resource "aws_vpc" "this" {
  cidr_block = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = var.name }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id
}

resource "aws_subnet" "public" {
  for_each = { for i, az in var.azs :
    az => var.public_subnet_cidrs[i] }
  vpc_id = aws_vpc.this.id
  availability_zone       = each.key
  cidr_block               = each.value
  map_public_ip_on_launch = true
}

resource "aws_subnet" "private" {
  for_each = { for i, az in var.azs :
    az => var.private_subnet_cidrs[i] }
  vpc_id             = aws_vpc.this.id
  availability_zone  = each.key
  cidr_block         = each.value
}

# One EIP + NAT Gateway PER AZ = real HA
# (an AZ never depends on another AZ's NAT)
resource "aws_eip" "nat" {
  for_each = aws_subnet.public
  domain   = "vpc"
}

resource "aws_nat_gateway" "this" {
  for_each      = aws_subnet.public
  allocation_id = aws_eip.nat[each.key].id
  subnet_id     = each.value.id
}
```

```
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }
}

resource "aws_route_table" "private" {
  for_each = aws_subnet.private
  vpc_id   = aws_vpc.this.id
  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.this[each.key].id
  }
}

resource "aws_route_table_association" "public" {
  for_each       = aws_subnet.public
  subnet_id      = each.value.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private" {
  for_each       = aws_subnet.private
  subnet_id      = each.value.id
  route_table_id = aws_route_table.private[each.key].id
}
```

Say out loud: "each AZ's private subnet routes through its own AZ's NAT Gateway — one AZ losing its NAT never affects another AZ."

Fubo Senior SRE interview — one-page reference · pairs with the full interactive prep guide